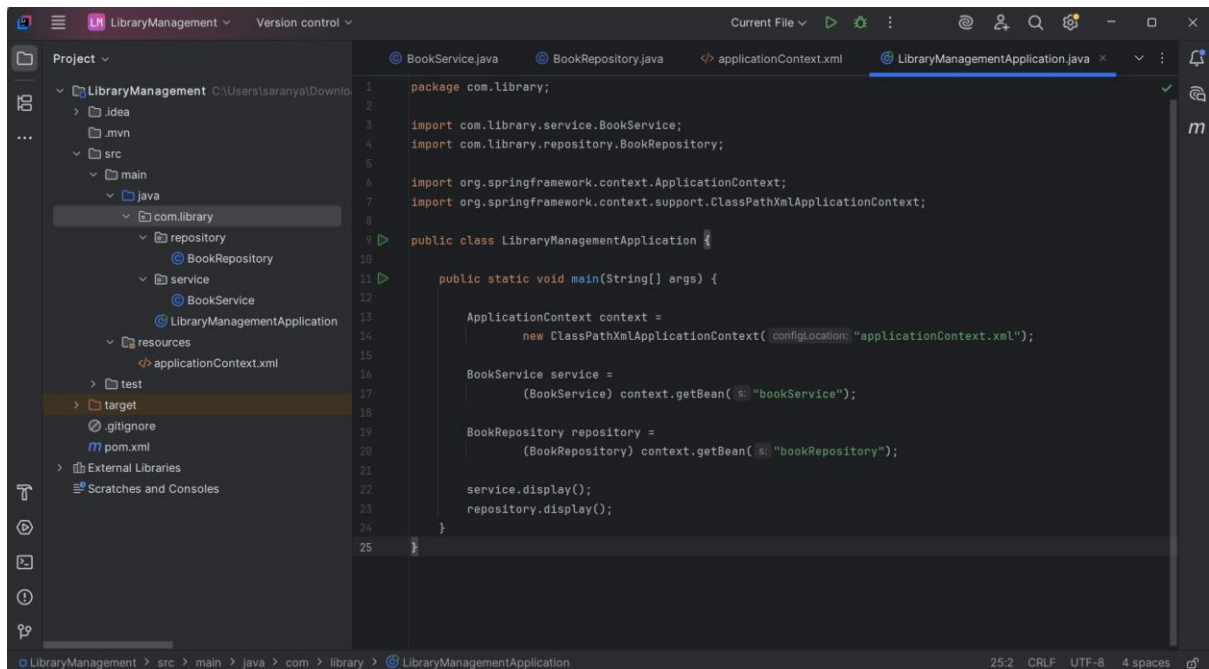


Spring Core and Maven Exercises

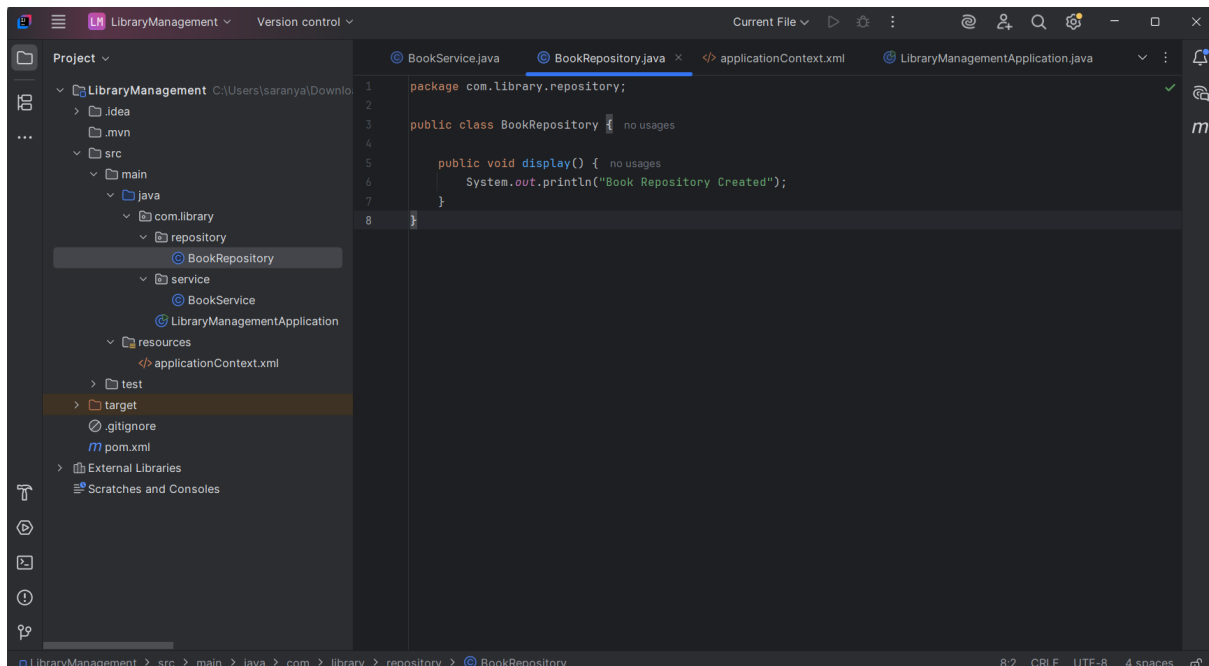
Exercise 1: Configuring a Basic Spring Application

Scenario:

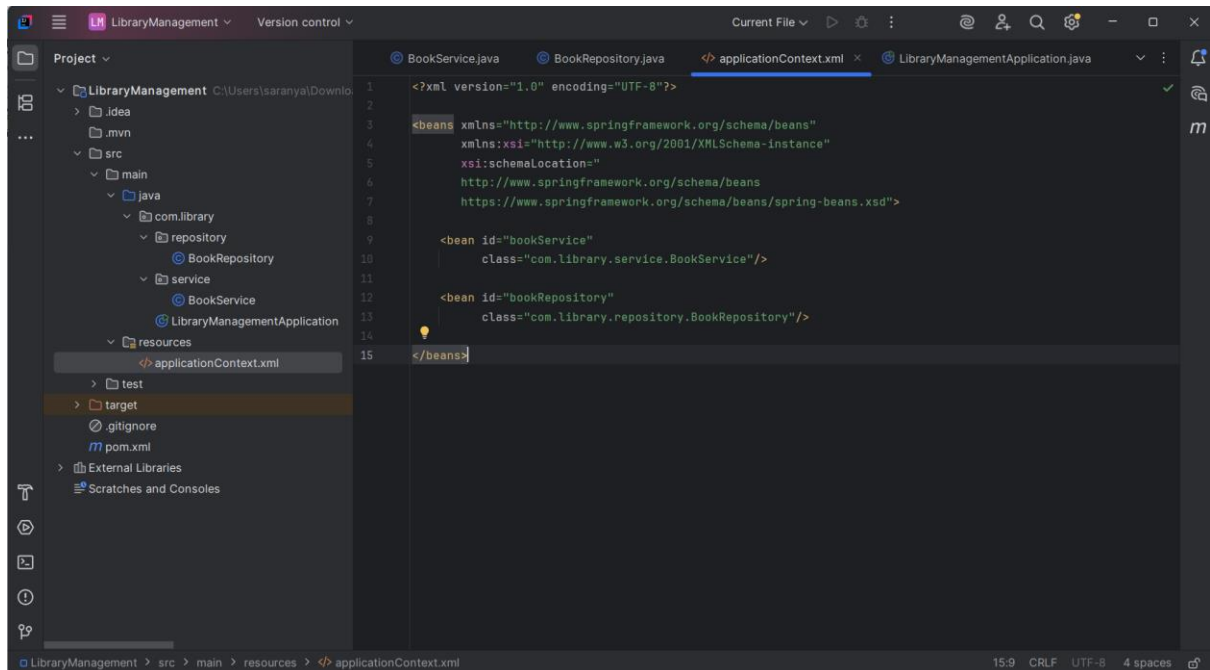
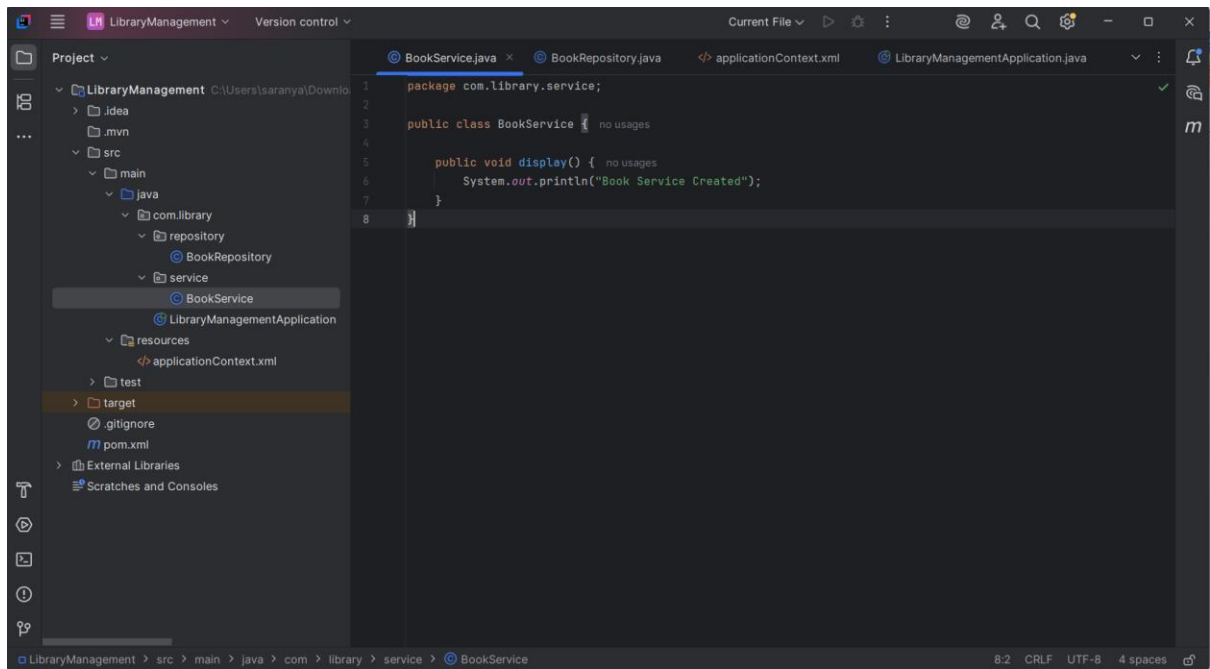
Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.



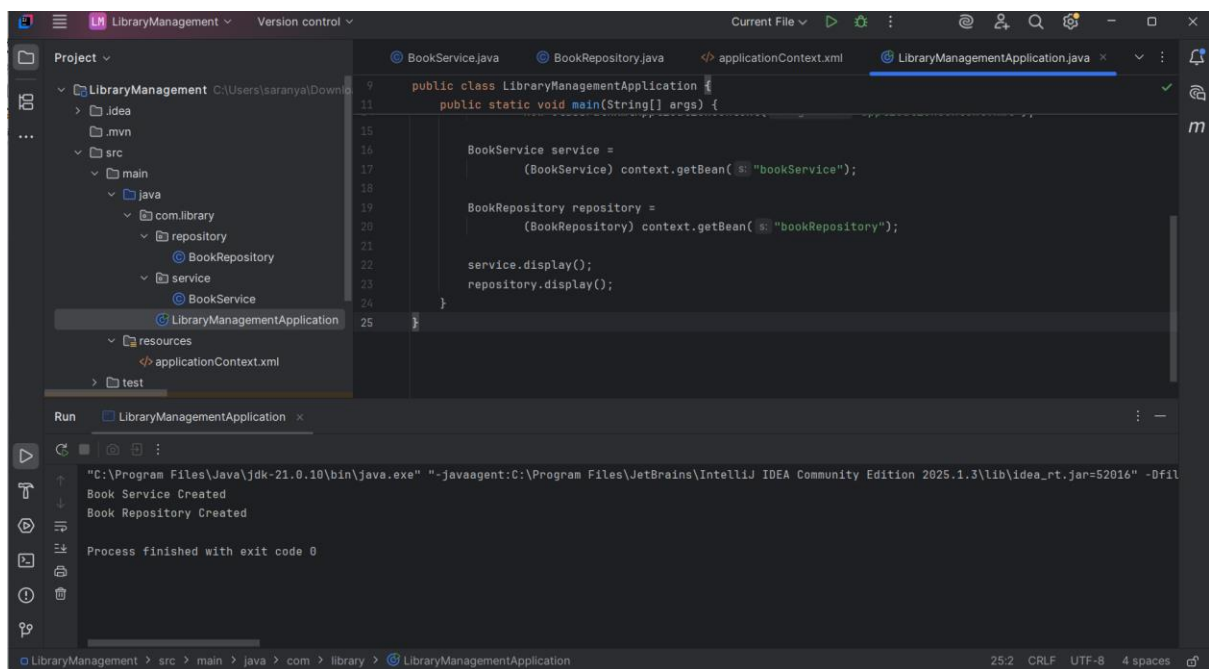
```
1 package com.library;
2
3 import com.library.service.BookService;
4 import com.library.repository.BookRepository;
5
6 import org.springframework.context.ApplicationContext;
7 import org.springframework.context.support.ClassPathXmlApplicationContext;
8
9 public class LibraryManagementApplication {
10
11     public static void main(String[] args) {
12
13         ApplicationContext context =
14             new ClassPathXmlApplicationContext("applicationContext.xml");
15
16         BookService service =
17             (BookService) context.getBean("bookService");
18
19         BookRepository repository =
20             (BookRepository) context.getBean("bookRepository");
21
22         service.display();
23         repository.display();
24     }
25 }
```



```
1 package com.library.repository;
2
3 public class BookRepository {
4
5     public void display() {
6         System.out.println("Book Repository Created");
7     }
8 }
```



OUTPUT:



```
public class LibraryManagementApplication {
    public static void main(String[] args) {
        BookService service =
            (BookService) context.getBean("bookService");

        BookRepository repository =
            (BookRepository) context.getBean("bookRepository");

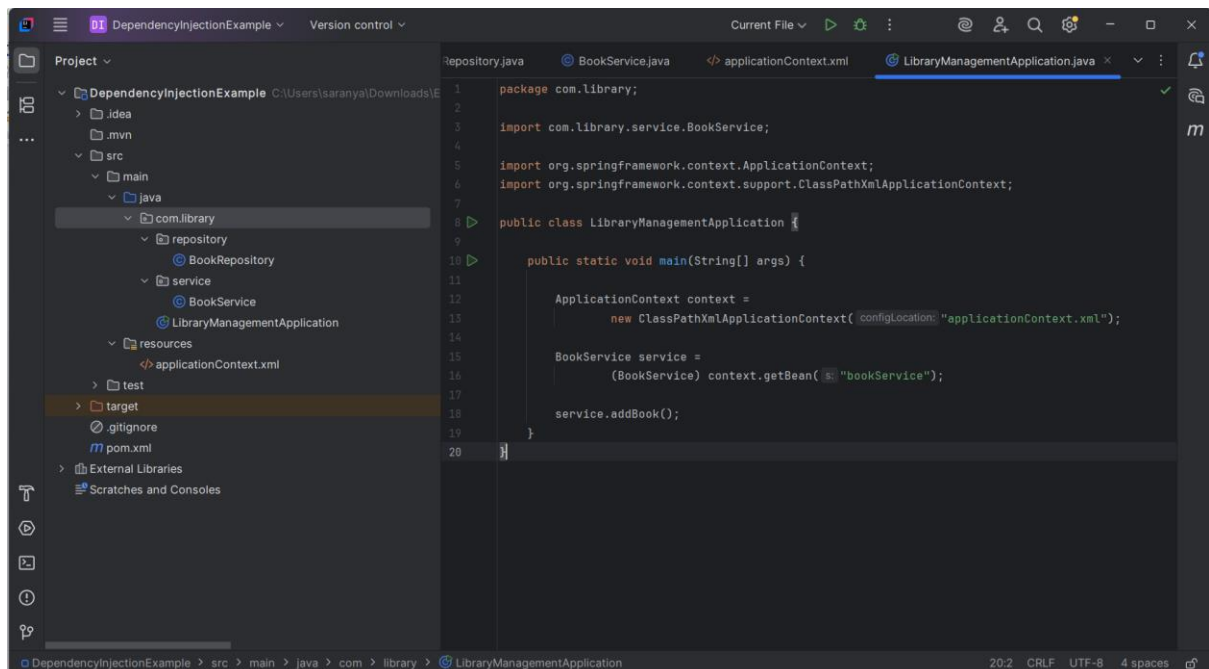
        service.display();
        repository.display();
    }
}
```

```
"C:\Program Files\Java\jdk-21.0.10\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3\lib\idea_rt.jar=52016" -Dfile.encoding=UTF-8
Book Service Created
Book Repository Created
Process finished with exit code 0
```

Exercise 2: Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the `BookService` and `BookRepository` classes using Spring's IoC and DI.



```
package com.library;

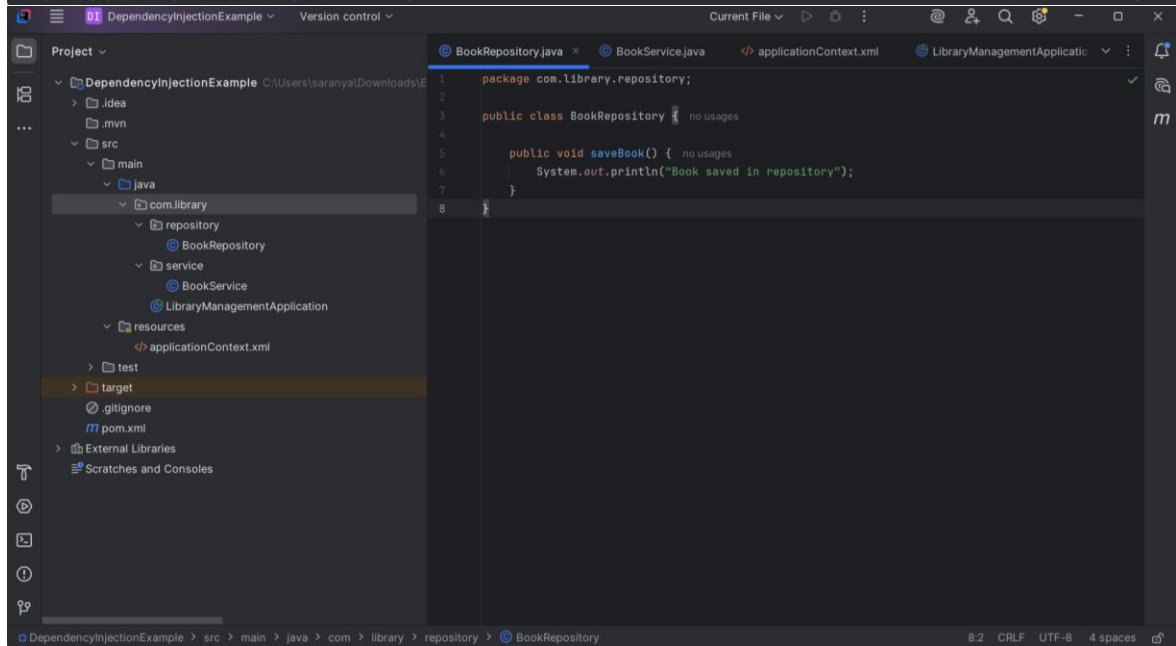
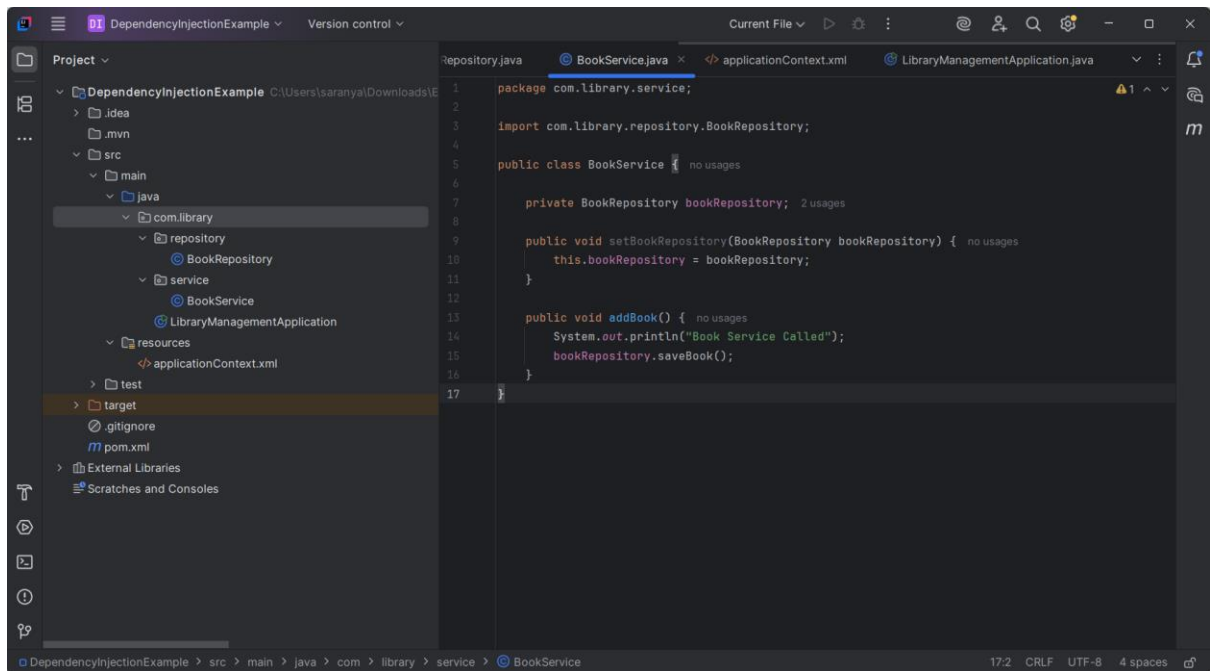
import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

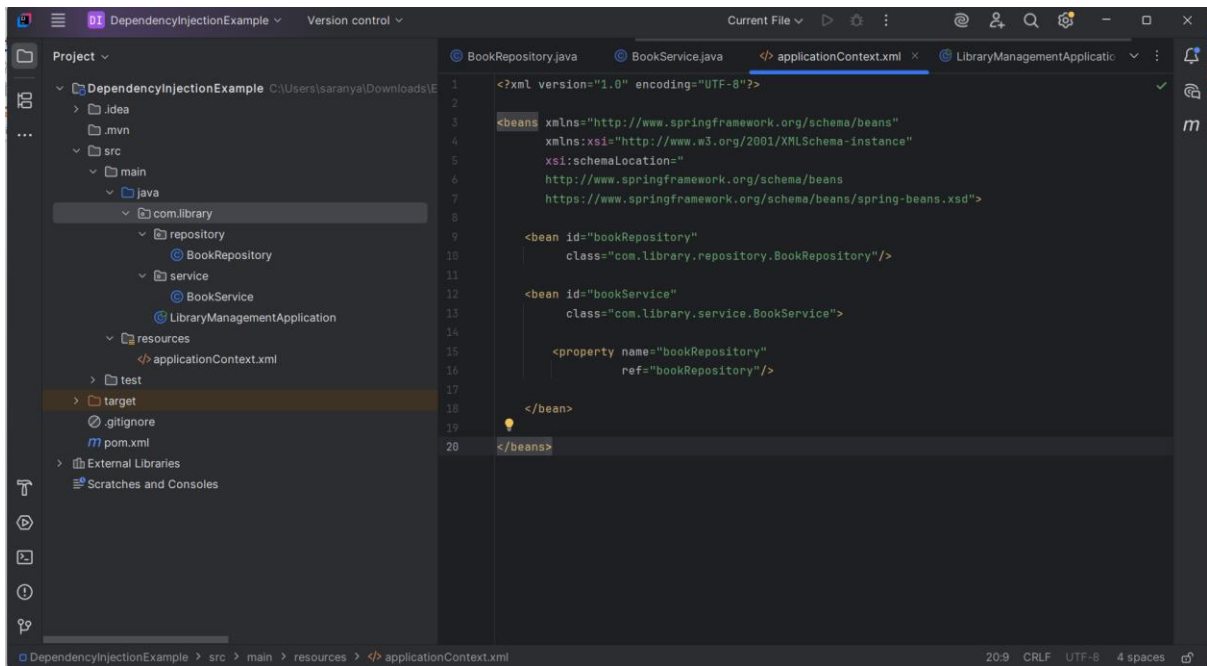
public class LibraryManagementApplication {

    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

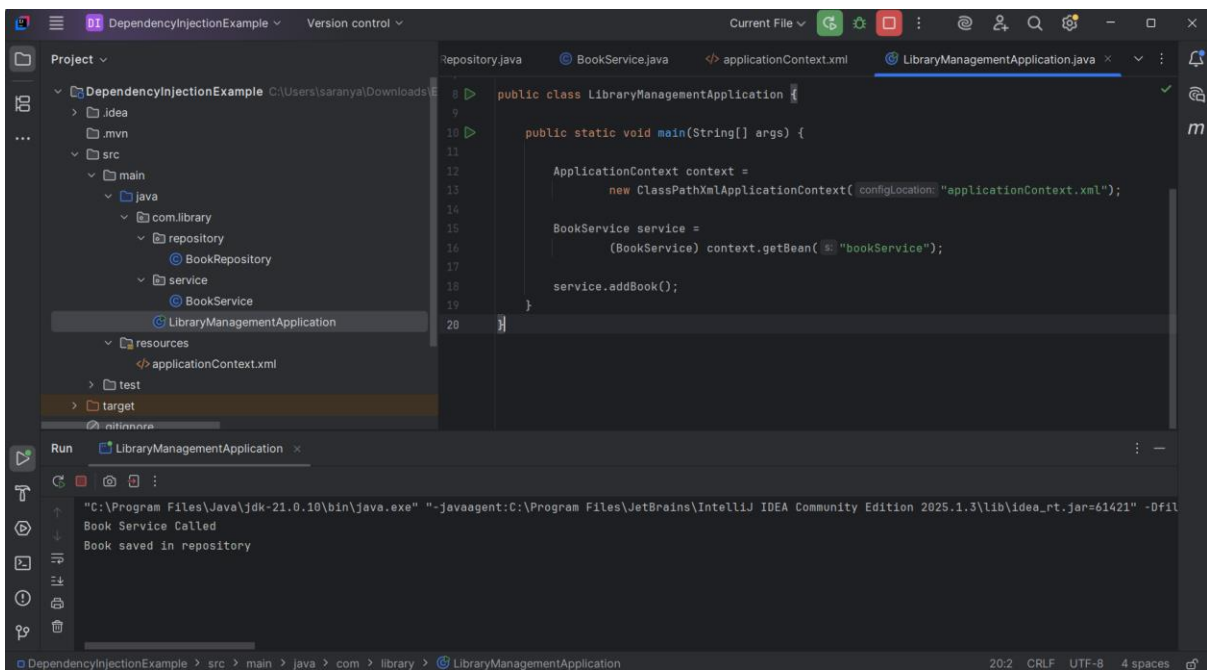
        BookService service =
            (BookService) context.getBean("bookService");

        service.addBook();
    }
}
```





OUTPUT



Exercise 4: Creating and Configuring a Maven Project

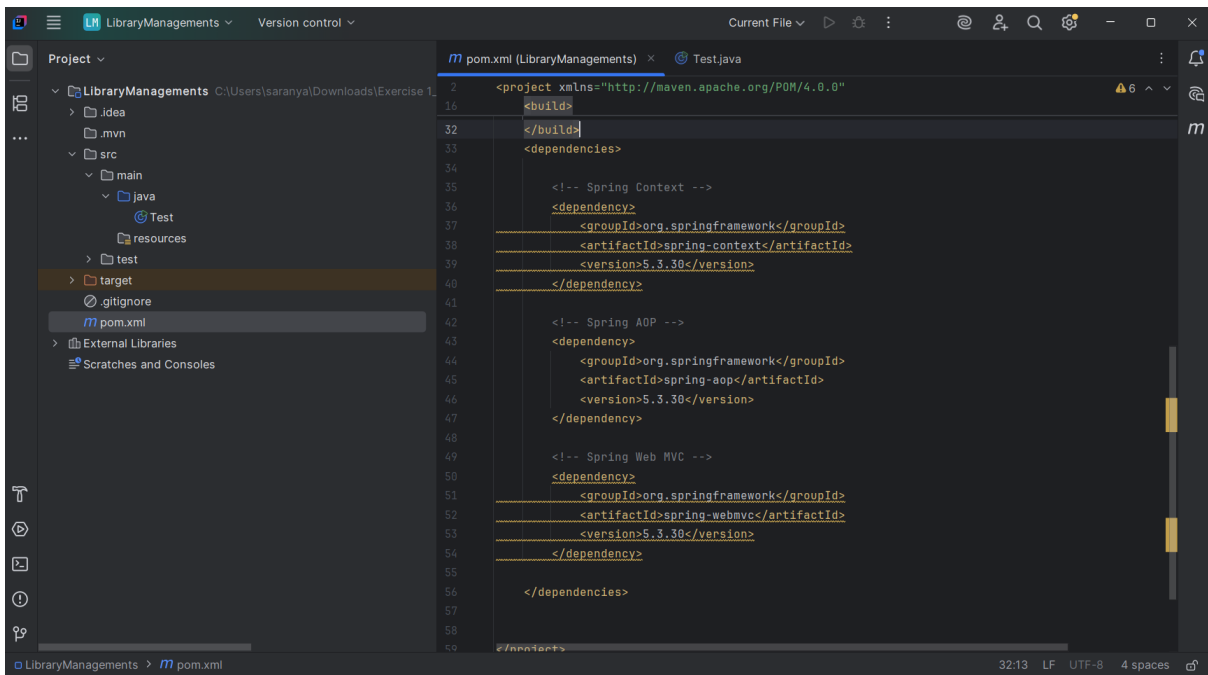
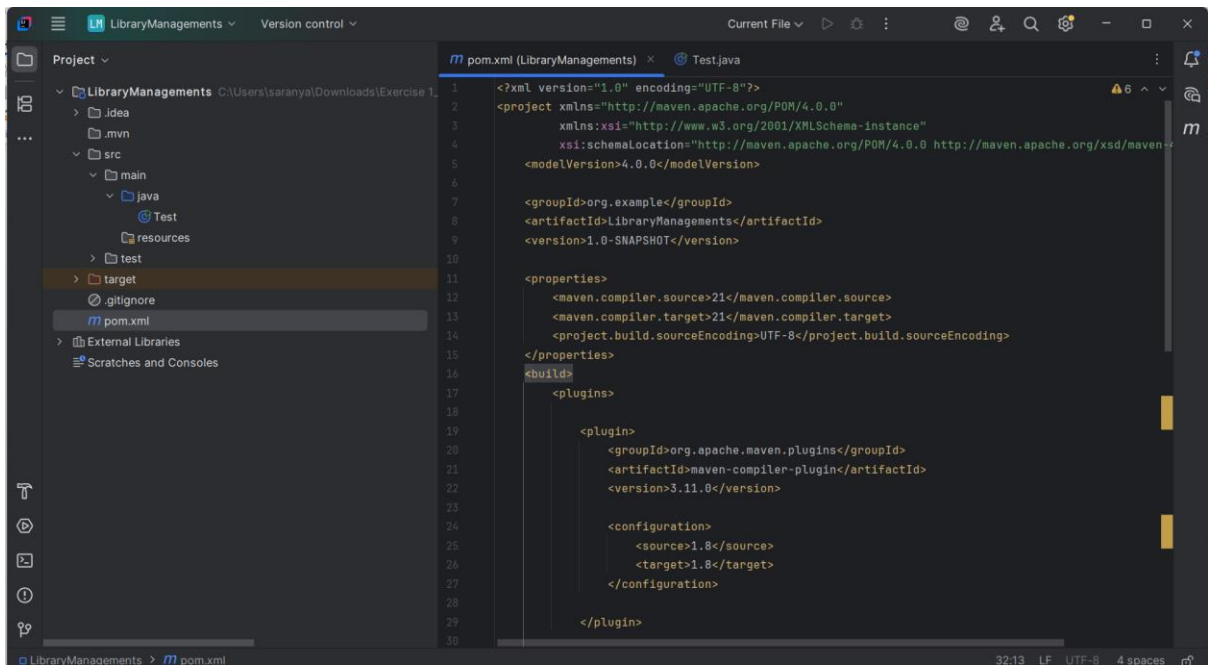
Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

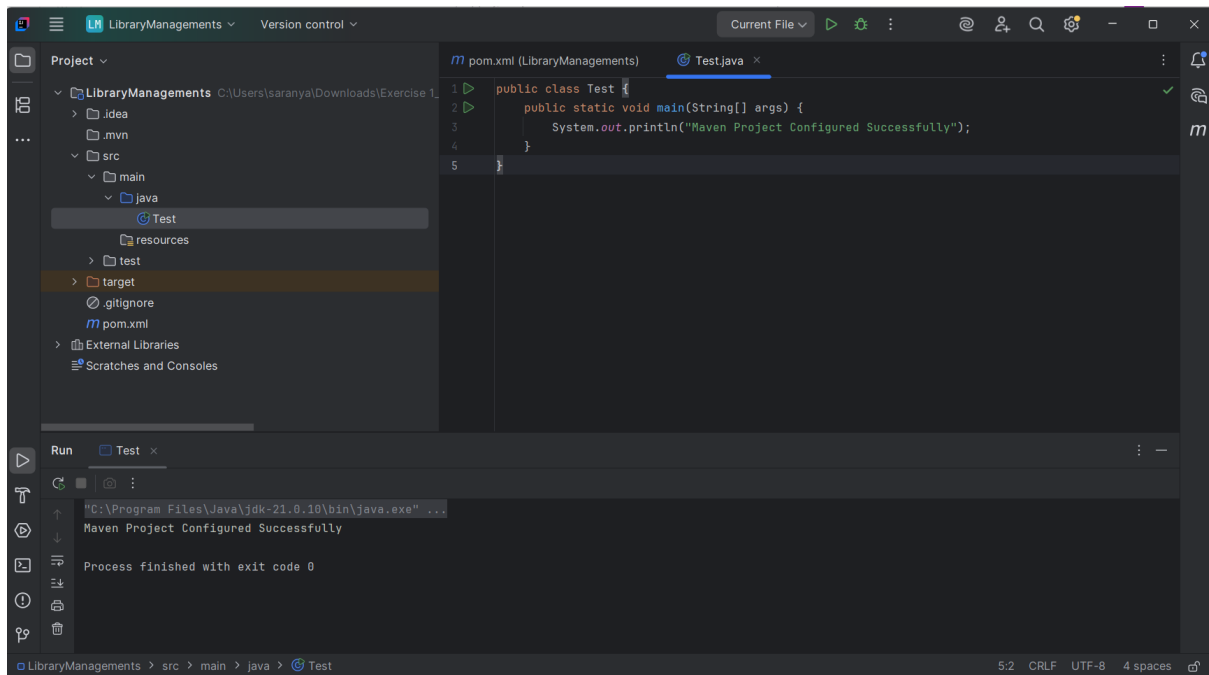
Project ▾



- ▾ LibraryManagements C:\Users\saranya\Downloads\Exercise 1
 - > .idea
 - .mvn
 - ▾ src
 - ▾ main
 - ▾ java
 - Test
 - resources
 - > test
 - > target
 - .gitignore
 - m pom.xml
- > External Libraries
- Scratches and Consoles



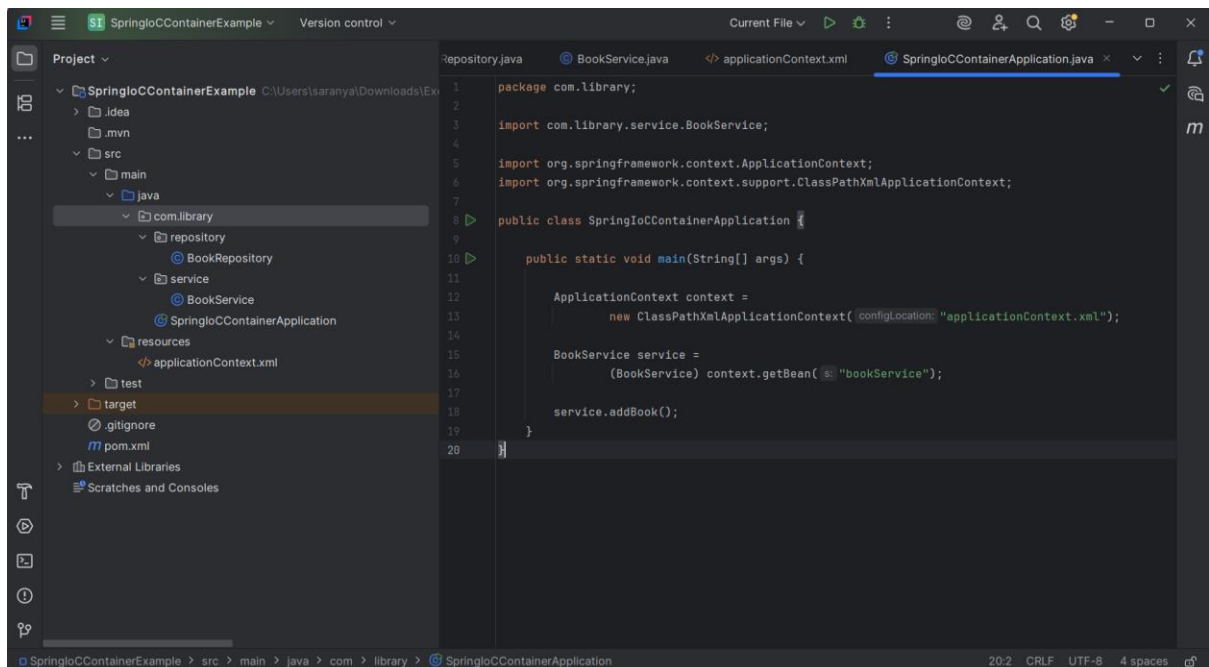
OUTPUT:

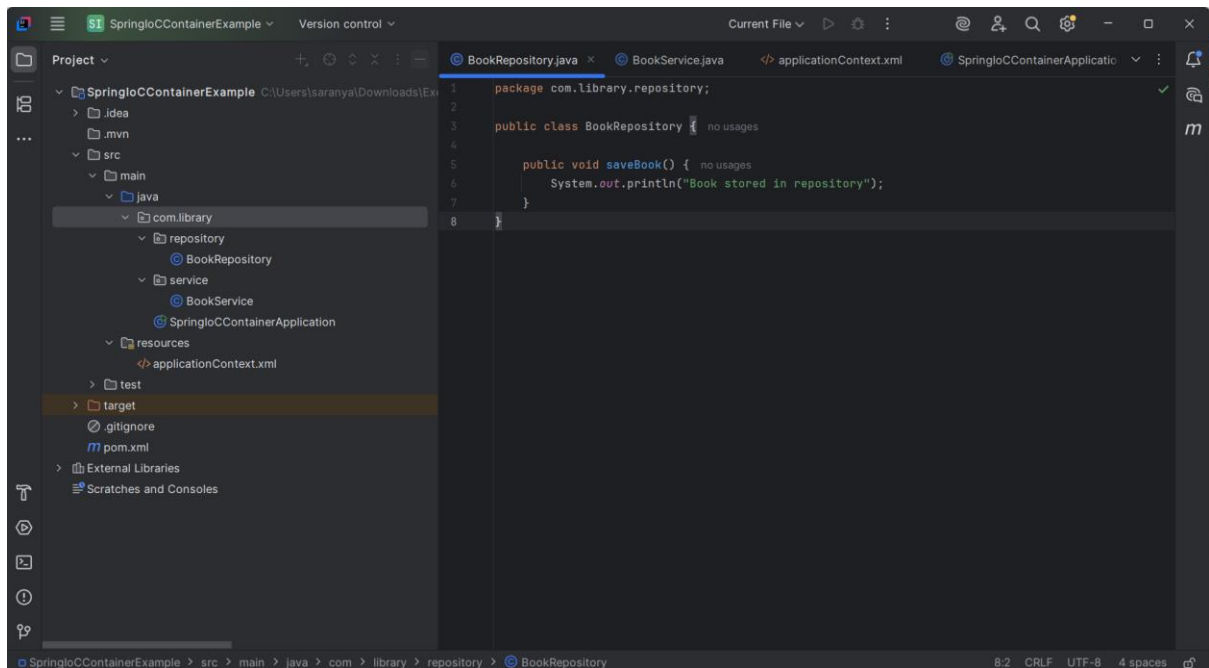
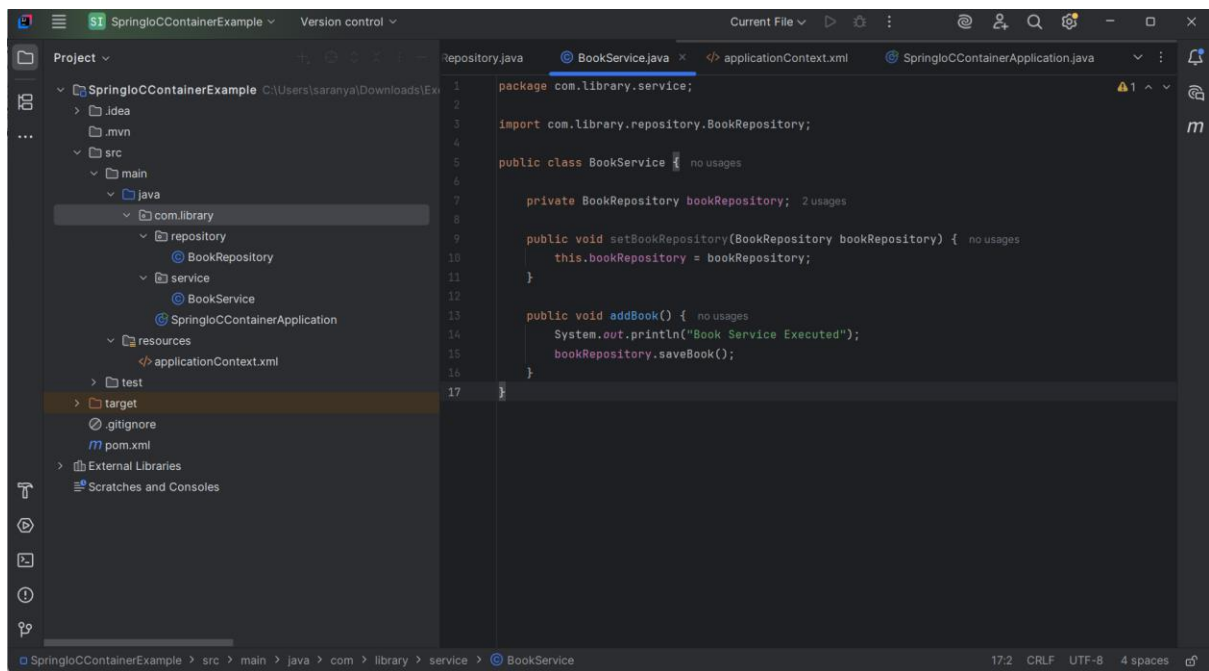


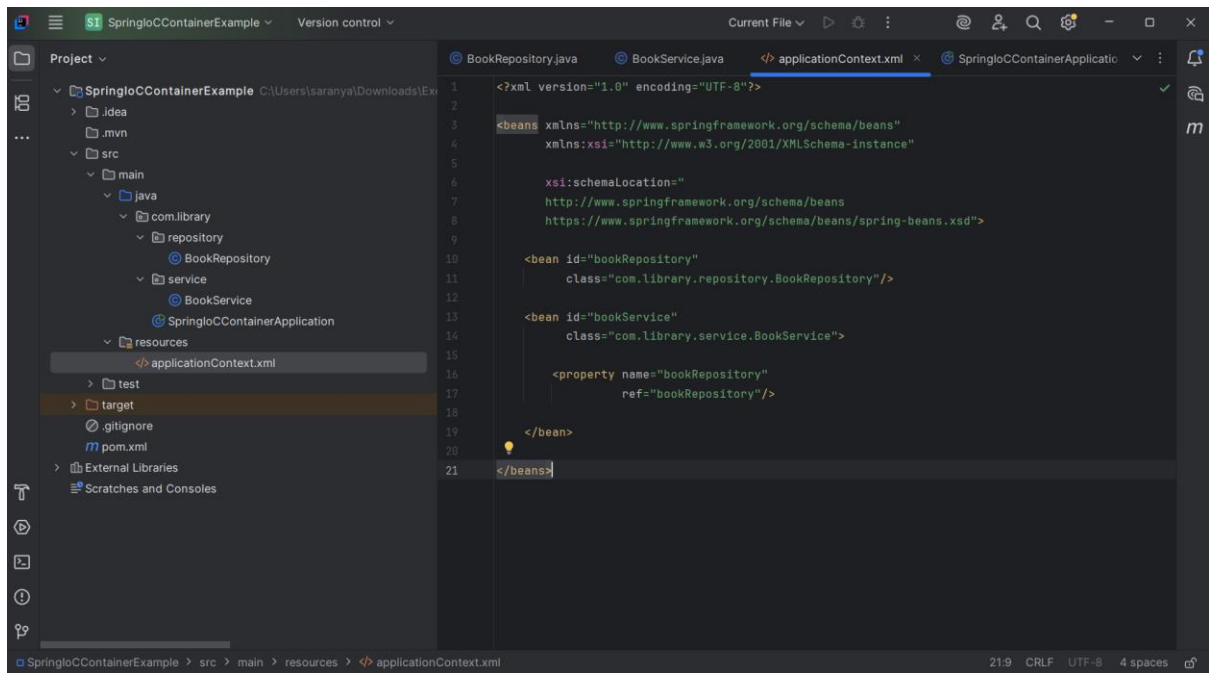
Exercise 5: Configuring the Spring IoC Container

Scenario:

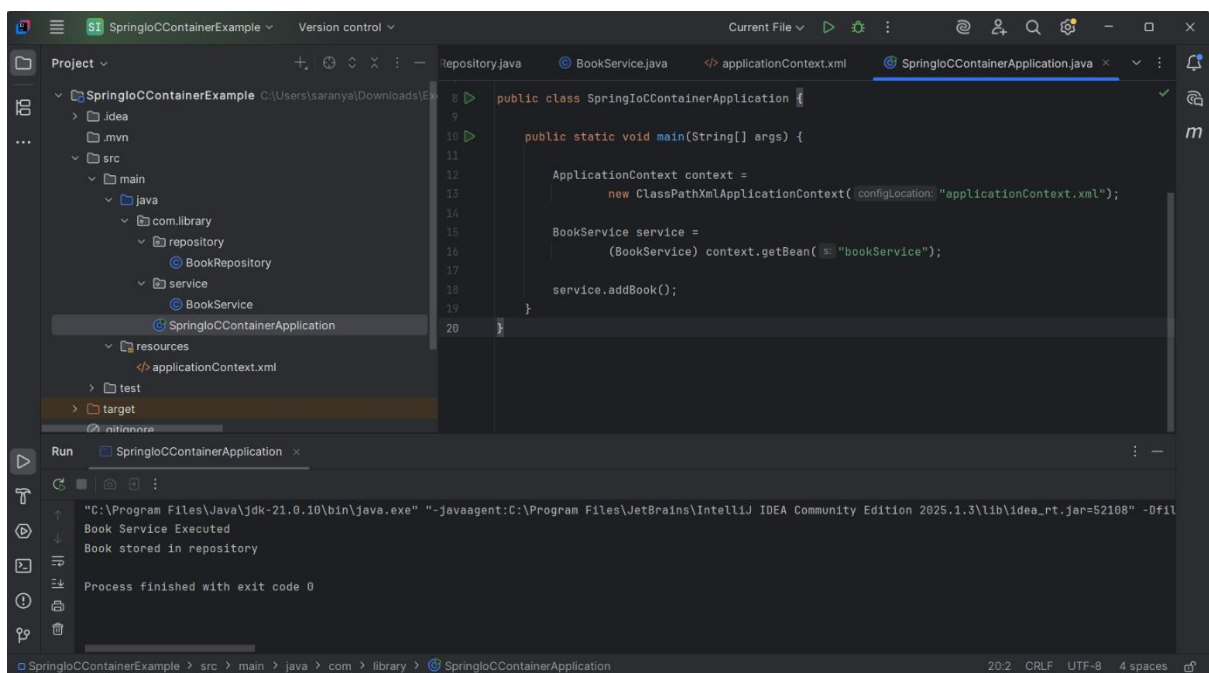
The library management application requires a central configuration for beans and dependencies.







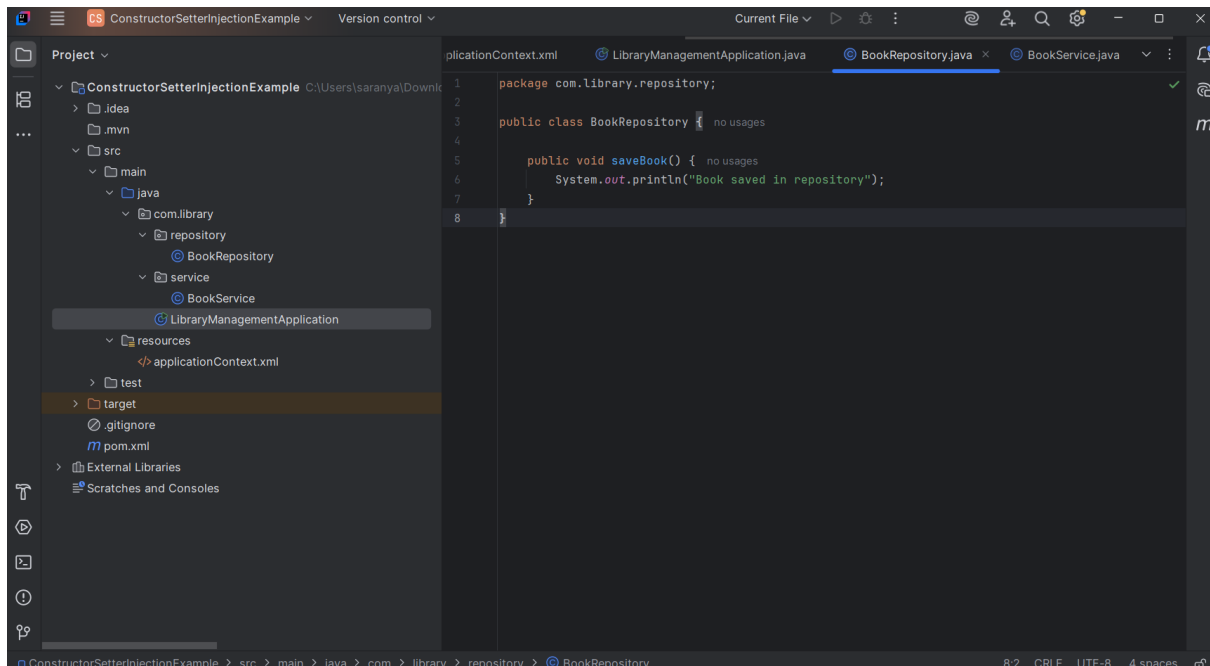
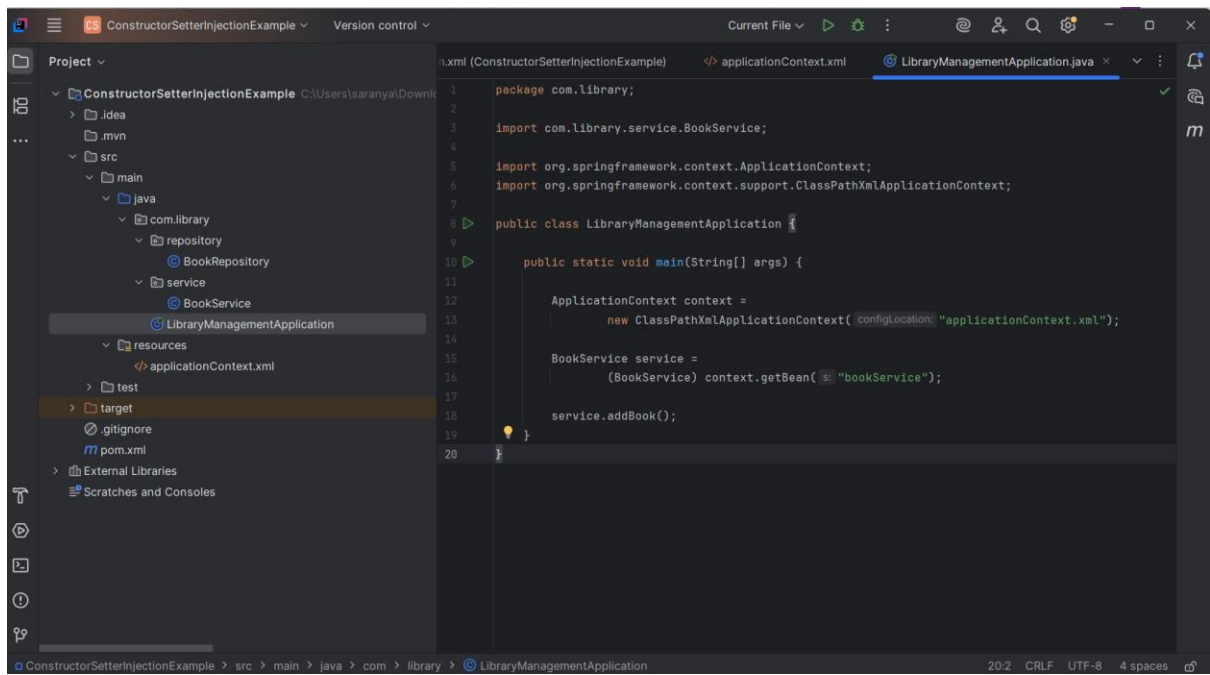
OUTPUT:

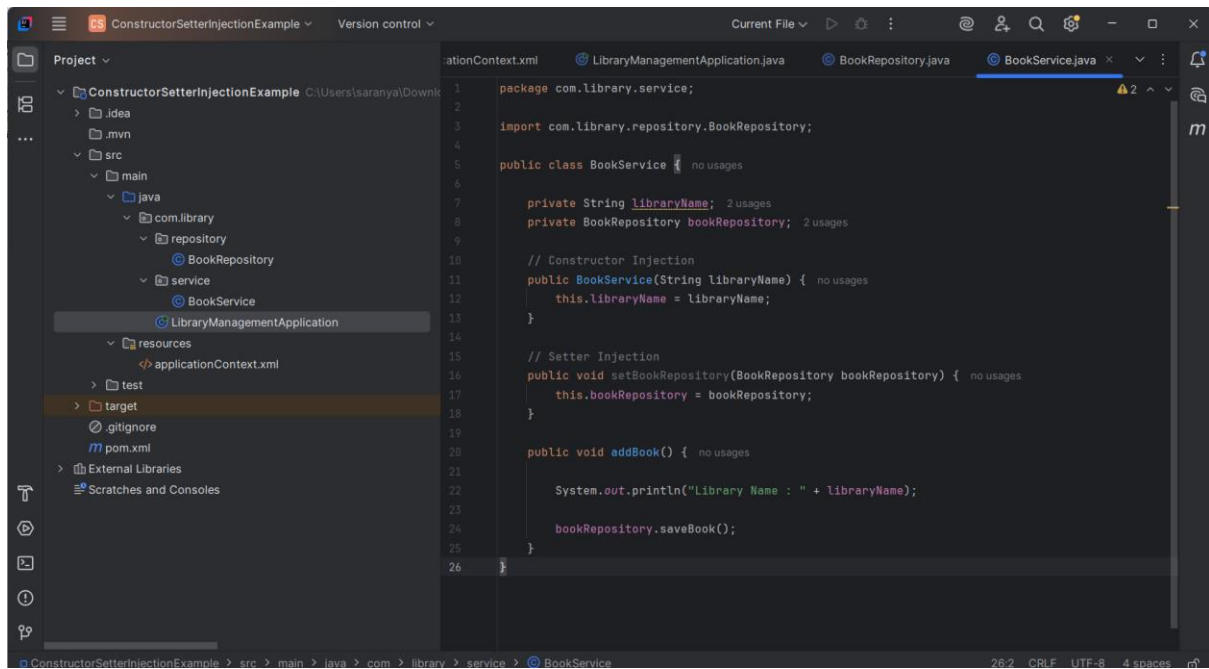


Exercise 7: Implementing Constructor and Setter Injection

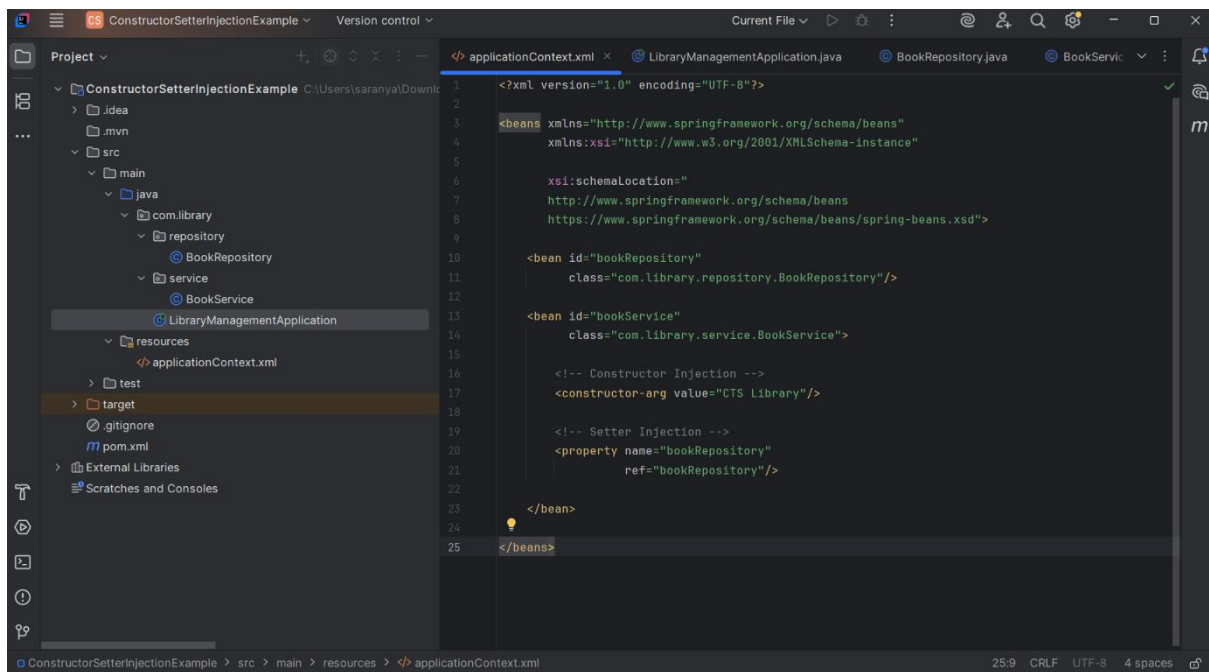
Scenario:

The library management application requires both constructor and setter injection for better control over bean initialization.



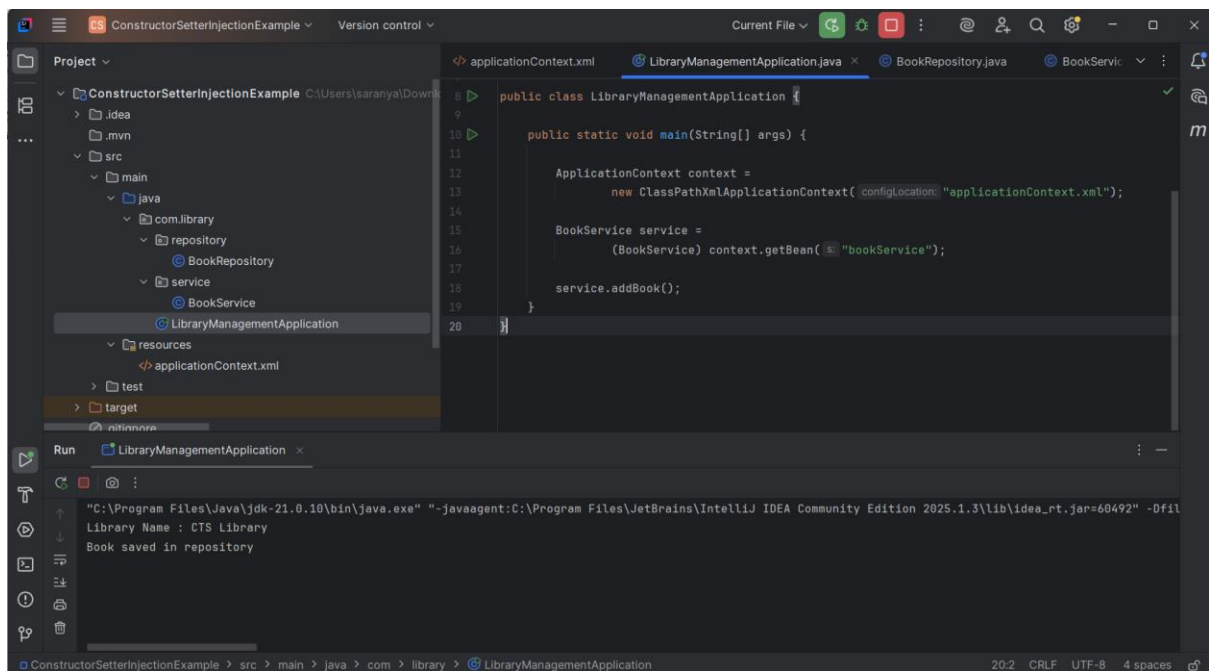


```
1 package com.library.service;
2
3 import com.library.repository.BookRepository;
4
5 public class BookService { no usages
6
7     private String libraryName; 2 usages
8     private BookRepository bookRepository; 2 usages
9
10    // Constructor Injection
11    public BookService(String libraryName) { no usages
12        this.libraryName = libraryName;
13    }
14
15    // Setter Injection
16    public void setBookRepository(BookRepository bookRepository) { no usages
17        this.bookRepository = bookRepository;
18    }
19
20    public void addBook() { no usages
21
22        System.out.println("Library Name : " + libraryName);
23
24        bookRepository.saveBook();
25    }
26 }
```



```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <beans xmlns="http://www.springframework.org/schema/beans"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5
6       xsi:schemaLocation="
7           http://www.springframework.org/schema/beans
8           https://www.springframework.org/schema/beans/spring-beans.xsd">
9
10    <bean id="bookRepository"
11          class="com.library.repository.BookRepository"/>
12
13    <bean id="bookService"
14          class="com.library.service.BookService">
15
16        <!-- Constructor Injection -->
17        <constructor-arg value="CTS Library"/>
18
19        <!-- Setter Injection -->
20        <property name="bookRepository"
21                  ref="bookRepository"/>
22
23    </bean>
24
25 </beans>
```

OUTPUT:

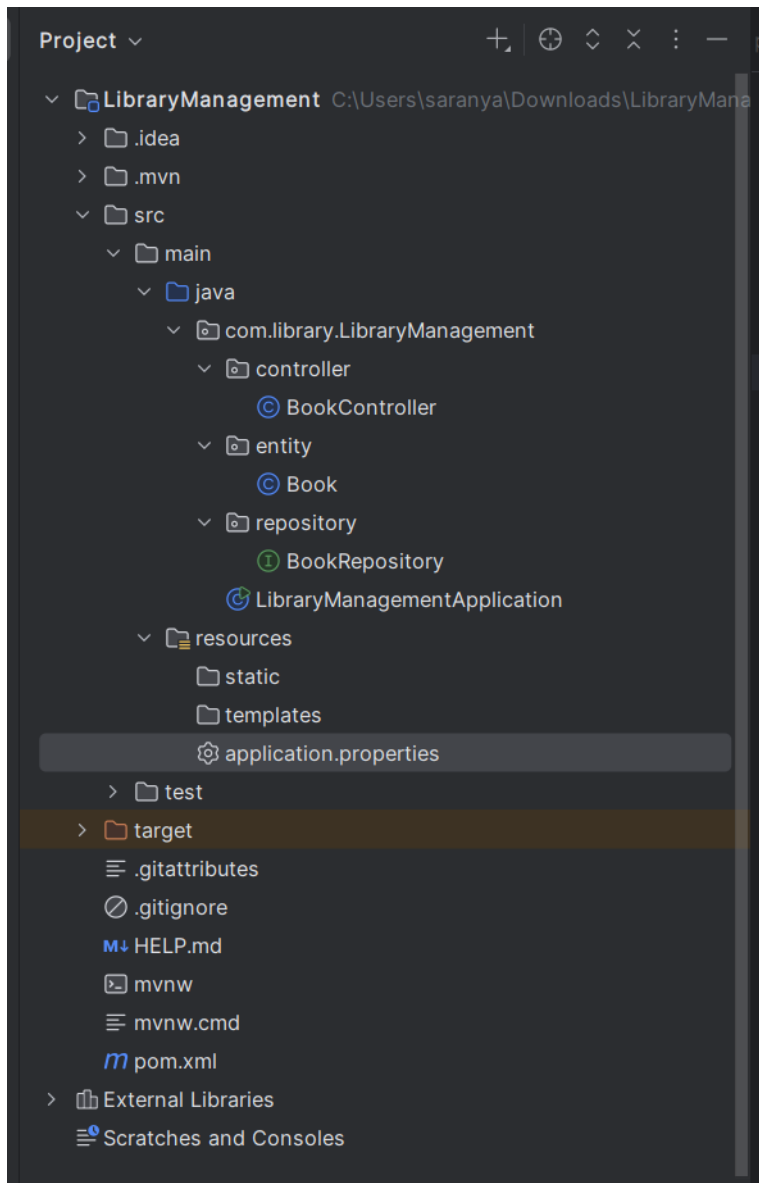


Exercise 9: Creating a Spring Boot Application

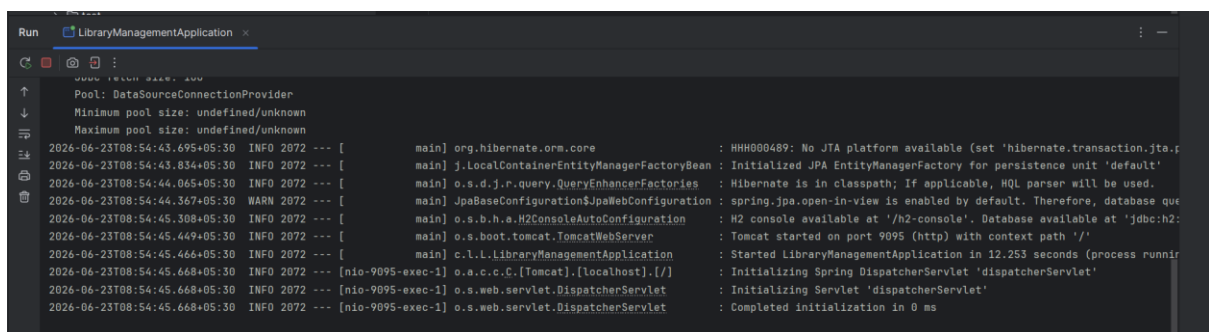
Scenario:

You need to create a Spring Boot application for the library management system to simplify configuration and deployment.

1. Project Structure



2. Spring Boot Application Running



3. Create Book 1

POST http://localhost:9095/books

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "id": 1,
3   "title": "Java Programming",
4   "author": "James Gosling"
5 }
```

Body Cookies Headers (5) Test Results Status: 200 OK Time: 340 ms Size: 216 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "title": "Java Programming",
4   "author": "James Gosling"
5 }
```

4. Create Book 2

POST http://localhost:9095/books

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "id": 2,
3   "title": "Spring Boot",
4   "author": "Craig Walls"
5 }
```

Body Cookies Headers (5) Test Results Status: 200 OK Time: 18 ms Size: 209 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 2,
3   "title": "Spring Boot",
4   "author": "Craig Walls"
5 }
```

5. Create Book 3

POST http://localhost:9095/books Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 {
2   "id": 3,
3   "title": "Effective Java",
4   "author": "Joshua Bloch"
5 }
```

Body Cookies Headers (5) Test Results Status: 200 OK Time: 14 ms Size: 213 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 3,
3   "title": "Effective Java",
4   "author": "Joshua Bloch"
5 }
```

200 OK

Standard response for successful HTTP requests. The actual response will depend on the request method used. In a GET request, the response will contain an entity corresponding to the requested resource. In a POST request the response will contain an entity describing or containing the result of the action.

6. Create Book 4

GET http://localhost:9095/books Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

Body Cookies Headers (5) Test Results Status: 200 OK Time: 17 ms Size: 445 B Save Response

Pretty Raw Preview Visualize JSON

```
2 {
3   "id": 1,
4   "title": "Java Programming",
5   "author": "James Gosling"
6 },
7 {
8   "id": 2,
9   "title": "Spring Boot",
10  "author": "Craig Walls"
11 },
12 {
13   "id": 3,
14   "title": "Effective Java",
15   "author": "Joshua Bloch"
16 },
17 {
18   "id": 4,
19   "title": "Clean Code",
20   "author": "Robert Martin"
21 },
22 {
23   "id": 5,
24   "title": "Head First Java",
25   "author": "Kathy Sierra"
26 }
```

7. Update Book

PUT http://localhost:9095/books/3 Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "title": "Effective Java Third Edition",
3   "author": "Joshua Bloch"
4 }
```

Body Cookies Headers (5) Test Results Status: 200 OK Time: 18 ms Size: 227 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 3,
3   "title": "Effective Java Third Edition",
4   "author": "Joshua Bloch"
5 }
```

8. Read All Books

GET http://localhost:9095/books Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

Body Cookies Headers (5) Test Results Status: 200 OK Time: 12 ms Size: 459 B Save Response

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "id": 1,
4     "title": "Java Programming",
5     "author": "James Gosling"
6   },
7   {
8     "id": 2,
9     "title": "Spring Boot",
10    "author": "Craig Walls"
11  },
12  {
13    "id": 3,
14    "title": "Effective Java Third Edition",
15    "author": "Joshua Bloch"
16  },
17  {
18    "id": 4,
19    "title": "Clean Code",
20    "author": "Robert Martin"
21  },
22  {
23    "id": 5,
24    "title": "Head First Java",
25    "author": "Kathy Sierra"
26  }
27 ]
```

9.Delete Book

DELETE

http://localhost:9092/books/2

Send

ParamsAuthorizationHeaders (8)BodyPre-request ScriptTestsSettings

noneform-datax-www-form-urlencodedrawbinaryJSON

1

BodyCookiesHeaders (5)Test Results

Status: 200 OKTime: 59 msSize: 189 BSave Response

PrettyRawPreviewVisualizeText

1 Book Deleted Successfully

10.Verify Delete

GET

http://localhost:9095/books

Send

ParamsAuthorizationHeaders (8)BodyPre-request ScriptTestsSettings

BodyCookiesHeaders (5)Test Results

Status: 200 OKTime: 15 msSize: 405 BSave Response

PrettyRawPreviewVisualizeJSON

```
1 {
2   {
3     "id": 1,
4     "title": "Java Programming",
5     "author": "James Gosling"
6   },
7   {
8     "id": 3,
9     "title": "Effective Java Third Edition",
10    "author": "Joshua Bloch"
11  },
12  {
13    "id": 4,
14    "title": "Clean Code",
15    "author": "Robert Martin"
16  },
17  {
18    "id": 5,
19    "title": "Head First Java",
20    "author": "Kathy Sierra"
21  }
22 }
```